1. THE PROBLEM STATEMENT (IN PLAIN WORDS)

Every second, modern organizational networks process millions of packets: logins, video calls
emails, and database queries. Concealed inside this legitimate traffic are malicious cyber-th
• Exploits: Attackers weaponizing bugs (buffer overflows, SQLi) to seize server control.
• Denial of Service (DoS): Volumetric connection floods designed to paralyze server operations
• Reconnaissance: Scanners (like Nmap) systematically mapping internal network ports.
• Backdoors & Shellcode: Stealth memory-injected payloads establishing remote command & contr

WHY TRADITIONAL DEFENSES (FIREWALLS & STATIC IDS) FAIL:
Traditional Intrusion Detection Systems (like classic Snort or static firewalls) rely on pre-
static signatures: "If packet payload contains exact hex string X from IP Y, block it."
Attackers trivially bypass this:
1. Polymorphic Malware: Changing a single byte in code alters the cryptographic hash complete
2. Payload Obfuscation & Dynamic Ports: Evades static port matching and keyword searches.
3. Zero-Day Vulnerabilities: Brand-new attack methods have no pre-existing signature in databa

THE AI BEHAVIORAL SOLUTION:
Instead of static text matching, AI analyzes traffic physics and behavioral telemetry:
• Connection duration, packet arrival frequency, and jitter (microsecond variances).
• Sent-to-received byte ratios and TCP handshake round-trip times.
• Time-To-Live (TTL) anomalies (e.g., standard Windows/Linux TTL vs. spoofed scanner TTL).
By learning baseline distributions of legitimate traffic vs. anomalous attack patterns,
AI detects novel, zero-day attacks in near real-time.

THE DUAL OPERATIONAL CHALLENGE:
• False Negatives (FN): Missed attack -> leads to data breaches, ransomware, or server takeov
• False Positives (FP): False alarm -> legitimate staff are blocked, creating a Self-Inflicte

CORE PROJECT OBJECTIVE:
Build, evaluate, and benchmark an end-to-end AI cyber threat detection pipeline comparing Classical Supervised ML (Logistic Regression, Decision Trees, Random Forests) against Deep Learning (Multi-Layer Perceptron), evaluating detection coverage (Recall), false alarms (FPR), training efficienc real-time inference latency, operational threshold tuning, and algorithmic dataset bias.

The UNSW-NB15 benchmark was synthesized by the Australian Cyber Security Centre (ACCS) using ar
IXIA PerfectStorm rig to generate 100 GB of real modern normal traffic and 9 distinct attack ty

FILE BREAKDOWN & PURPOSE:
1. UNSW_NB15_training-set.csv (175,341 records | ~32.1 MB)
   • Role: Used strictly to train models and compute preprocessor parameters (scaler mean/varia
   • Distribution: Normal Traffic = 56,000 (31.9%) | Malicious Traffic = 119,341 (68.1%).

2. UNSW_NB15_testing-set.csv (82,332 records | ~15.3 MB)
   • Role: Untouched blind evaluation benchmark. Kept strictly isolated during training.
   • Distribution: Normal Traffic = 37,000 (44.9%) | Malicious Traffic = 45,332 (55.1%).

3. UNSW-NB15_features.csv (44 Rows | ~3.2 KB)
   • Role: Feature metadata dictionary defining data types (float, integer, categorical) and gr

WHY ARE THERE 3 DATASETS INSTEAD OF JUST ONE?
• Academic Standardization: Pre-defined train/test splits ensure all international research
  compares models on the identical blind test set rather than differing random splits.
• Preventing Data Snooping: Preserves temporal integrity. Models cannot leak future packet stat
  into past training distributions.
• Feature Dictionary: Normalizes the technical schema across complex packet fields.

NETWORK FEATURES EXPLAINED IN SIMPLE TERMS (42 INPUT FEATURES):
1. Basic Features (dur, proto, service, state, spkts, dpkts, sbytes, dbytes, rate, sttl, dttl)
   • Quantifies connection duration, protocol (TCP/UDP), port service, packet and byte counts.
   • KEY INDICATOR: 'sttl' (Source Time-To-Live). Normal systems use 64 or 128. Automated attac
     scanners frequently exhibit extreme, anomalous TTL values (e.g. 254).

2. Content & Payload Features (sload, dload, sloss, dloss, swin, dwin, smean, dmean)
   • Measures bits per second, packet drops (sloss), TCP window sizes, and mean packet sizes.
   • Detects buffer overflows (abnormally large smean) and DoS flooding (massive loss & load).

3. Time & Jitter Features (sinpkt, dinpkt, sjit, djit, tcprtt, synack, ackdat)
   • Measures microsecond delays between packets and 3-way handshake round-trip times.
   • Distinguishes irregular, bursty human traffic from automated, rigidly timed botnet attacks

4. Generated Flow Features (ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm, ct_srv_dst)
   • Rolling historical connection counts tracking same IP/service across the last 100 flows.
   • Primary indicator for Port Scanning: 1 IP probing 90 different ports in 100 ms.

```
DATA PREPROCESSING: WHAT WAS DONE VS. WHAT WAS NOT DONE (AND WHY)
-----------------------------------------------------------------------------------------------
[✓] DONE: Drop 'id' column
    -> Reason: 'id' is an arbitrary sequential counter. Leaving it in causes "shortcut learning
       where the model predicts attacks based on row index rather than packet physics.

[✓] DONE: Isolate 'attack_cat' from input feature matrix (X)
    -> Reason: 'attack_cat' explicitly labels the attack name ("DoS", "Exploit"). Leaving it in
       causes target leakage. Preserved exclusively for Task 5 per-attack ethical bias auditing

[✓] DONE: Median Imputation for missing values
    -> Reason: Network telemetry has extreme positive skew (a few flows send 10M bytes; most
       send 500). Median prevents extreme outlier flows from distorting imputed values.

[✓] DONE: LabelEncoder on Categoricals ('proto', 'service', 'state')
    -> Reason: Converts string categories to consistent integers for linear and tree models.

[✓] DONE: StandardScaler (Fit on Train ONLY -> Transform Test)
    -> Reason: Prevents gradient explosion in Deep Learning & Logistic Regression. Crucially,
       mean (μ) and variance (σ) are calculated solely on training data to prevent test leakage

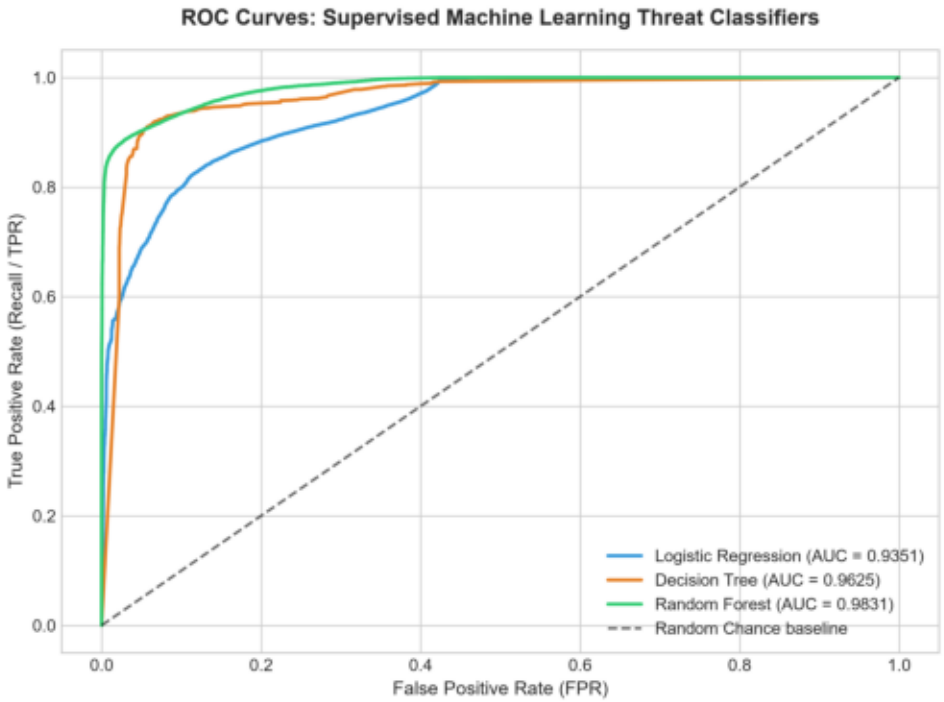
[✗] NOT DONE: Synthetic Oversampling (SMOTE)
    -> Reason: Synthetic packet generation distorts real network physics. Real enterprise netwo
       have rare attacks and abundant normal traffic; fake packets create massive false alarms

[✗] NOT DONE: Retrospective Re-splitting of Train/Test sets
    -> Reason: Merging and shuffling would violate the official benchmark specification and
       artificially inflate test accuracy by leaking temporal packet patterns.

THE 9 CYBER THREAT CATEGORIES IN THE DATASET:
1. Generic (33.6%): Cryptographic collision attacks and generic cipher evasion techniques.
2. Exploits (25.4%): Known vulnerability exploits targeting unpatched software servers.
3. Fuzzers (13.8%): Randomized, malformed packet bursts attempting to trigger buffer overflows
4. DoS (9.3%): Volumetric connection flooding targeting service availability.
5. Reconnaissance (8.0%): Port scanning (Nmap) and network topology discovery probes.
6. Analysis (1.5%): Vulnerability scanning (port scans, web directory brute-forcing).
7. Backdoor (1.3%): Covert command & control channels bypassing inbound firewall rules.
8. Shellcode (0.9%): Executable binary payloads injected into vulnerable memory addresses.
9. Worms (0.1%): Self-propagating autonomous malware (rarest attack category).
```

```
+-----------------------------------------------------------------------------------------------------+
|                                END-TO-END SYSTEM PIPELINE ARCHITECTURE                              |
+-----------------------------------------------------------------------------------------------------+
  [data/UNSW_NB15_training-set.csv] (175,341)          [data/UNSW_NB15_testing-set.csv] (82,332)
                              |                                          |
  [src/data_preprocessing.py]                                            |
  - Drop 'id' & isolate 'attack_cat'                                     |
  - LabelEncode ('proto', 'service', 'state')                            |
  - Fit StandardScaler on Train ONLY (μ, σ)                              |
                              |                                          |
                              ▼                                          ▼
                    Transformed X_train                        Transformed X_test
                              |                                          |
                              |                                          |
          ┌──────────────────┴──────────────────┐                       |
          ▼                                     ▼                       |
  [Classical ML Models]              [Deep Learning MLP]                 |
  - Logistic Regression              - Layer 1: 128 ReLU                 |
  - Decision Tree (Depth=12)         - Layer 2: 64 ReLU                  |
  - Random Forest (100 Trees)        - Output: 1 Sigmoid (Adam)          |
          |                                     |                       |
          └──────────────────┬──────────────────┘                       |
                             ▼                                          ▼
```

Figure 1: ROC Curves for Supervised Machine Learning Models (AUC Benchmark)



EMPIRICAL BENCHMARK RESULTS (EVALUATED ON 82,332 UNTOUCHED TEST PACKETS):

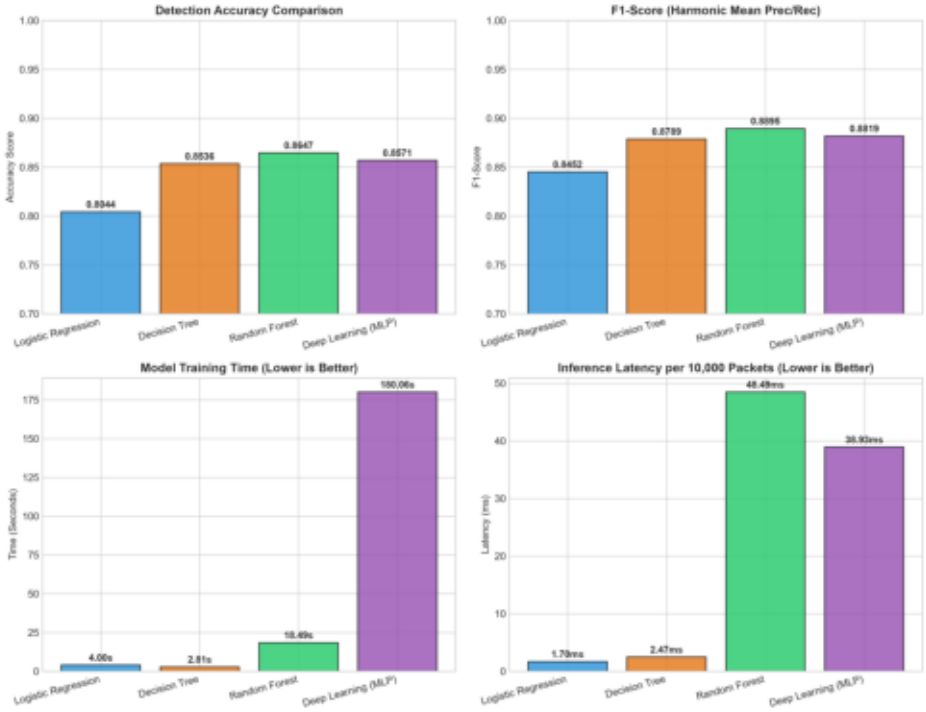
Model Architecture	Accuracy	Precision	Recall (Coverage)	F1-Score	ROC-AUC	FPR	Latency(1
Logistic Regression	80.44%	74.91%	96.95%	0.8452	0.9351	39.78%	1.70
Decision Tree (Depth=12)	85.36%	80.72%	96.46%	0.8789	0.9625	28.23%	2.47
Random Forest (Ensemble)	86.47%	80.81%	98.92%	0.8895	0.9831	28.78%	48.49
Deep Learning (MLP 128x64)	85.71%	80.89%	96.94%	0.8819	0.9686	28.06%	38.93

CHAMPION MODEL: Random Forest achieved 98.92% Recall (missed only 1.08% of threats) & 0.9831 ROC-AUC

WHY COMPARE ML VS DL? WHAT IS THE BENEFIT?

1. Structured Tabular Reality: Tree ensembles (Random Forest) systematically outperform Neural Networks on structured network flow tables (Accuracy 86.47% vs. 85.71%; Recall 98.92% vs. 96.94%)
2. Training Cost Penalty: Decision Tree trained in 2.8s; Random Forest in 18.5s; while Deep Learning MLP required 180.1s (60x slower) with zero accuracy gain.
3. Edge Firewall Deployment: Network firewalls (Cisco, Fortinet) run embedded CPUs. Decision Trees evaluate in 2.47 ms per 10k packets using simple IF/ELSE branches. Deep Learning requires floatin point matrix multiplication, which chokes low-power line-rate gateways.
4. SOC E works are black boxes.

Figure 2: Empirical Benchmark - Accuracy, F1, Training Time & Latency



THE FALSE ALARM CRISIS AT DEFAULT THRESHOLD (τ = 0.50):
At τ = 0.50, the False Positive Rate is 28.78%. Out of 37,000 legitimate normal sessions in the test set, 10,648 real users are blocked! In a hospital or university, shutting down 28% of innocent users causes operational paralysis and analyst alert fatigue.

OPERATIONAL THRESHOLD TUNING (τ = 0.82):
By shifting the classification probability threshold from τ = 0.50 -> 0.82:
• False Positive Rate drops from 28.78% down to 5.57% (a 5x reduction in false alarms!).
• Precision jumps from 80.81% to 95.22% (analysts can trust that alerts are genuine).
• Security Detection Coverage remains high at 90.67% Recall.

DATASET BIAS AUDIT (PER-ATTACK RECALL ON TEST SET):
• Generic: 100.0% (18,871/18,871)
• Exploits: 99.69% (11,098/11,132)
• DoS: 99.90% (4,085/4,089)
• Fuzzers (The Vulnerable Class): 92.73% (441 missed packets due to mutated payload mimicry).
• Reconnaissance: 99.94% (3,494/3,496)
• Backdoor: 100.0% (583/583)
• Worms: 100.0% (44/44)

COMPLETE REPOSITORY FILE GUIDE:
• src/main.py: Master orchestrator executing Tasks 1-5 end-to-end with a single command.
• src/data_preprocessing.py: Implements load_data(), median fillna, LabelEncoder & StandardScaler.
• src/eda_analysis.py: Generates threat distributions, protocol plots, KDE curves & feature importan
• src/ml_models.py: Trains Logistic Regression, Decision Tree & Random Forest with ROC evaluation.
• src/dl_models.py: Multi-Layer Perceptron (128x64 units) with Adam optimizer & comparison plots.
• src/ethical_sim.py: Simulates threshold trade-offs & audits per-attack detection bias.
• src/validate_env.py: Diagnostic script verifying Python, dependencies, and dataset integrity.
• notebooks/UNSW_NB15_Analysis.ipynb: Interactive lab notebook with code, outputs & embedded charts.
• outputs/best_threat_detector_rf.joblib: Serialized Random Forest model ready for live deployment.
• report/COMPREHENSIVE_PROJECT_REPORT.html: Styled offline HTML report.
• report/AI_Cybersecurity_Unit1_Comprehensive_Report.pdf: This native multi-page PDF document.